

PA4 Flash Attention in CUDA

In this assignment, you will implement scaled dot-product attention in CUDA and optimize it using the **FlashAttention** algorithm. Before you begin, read through both the lecture slides and the original FlashAttention paper carefully. Please check the following points before starting the assignment:

- All four implementations must produce **numerically consistent** outputs. For a fixed random input, the maximum absolute difference between any two implementations must be $< 10^{-3}$.
- Always call `torch.cuda.synchronize()` before and after timing loops to flush all pending GPU work.
- Use `float16` tensors on the GPU throughout Parts A–C.
- Run each timing measurement for **at least 10 iterations** and calculate the average.

Part A: Standard Attention in PyTorch

Implement standard scaled dot-product attention using PyTorch tensor operations. This is your correctness baseline and the performance reference against which FlashAttention will be compared.

Given the input tensors $Q, K, V \in \mathbb{R}^{B \times N_H \times N \times D}$, compute:

$$O = \text{softmax}\left(\frac{QK^\top}{\sqrt{D}}\right)V$$

Your implementation must:

- Use `torch.matmul` for both matrix multiplications.
- Use `torch.softmax(..., dim=-1)` along the key dimension.
- Form the full $N \times N$ attention matrix explicitly; do *not* tile or use any loops over blocks.
- Batch size $B = 8$, number of heads $N_H = 16$, head dimension $D = 64$.
- Vary the sequence length $N \in \{32, 64, 128, 256, 512, 1024, 2048, 4096, 8192\}$.
- Use `torch.float16` tensors on `device='cuda'`.
- Measure the average runtime over ≥ 10 iterations; plot runtime (ms) vs. N .

Part B: FlashAttention Algorithm in PyTorch

Implement the FlashAttention algorithm as described in the paper using Python `for` - loops over tiles. The goal is **not performance**; Python-level tiling will be much slower than Part A; but it aims to build an understanding of the online softmax update before porting to CUDA.

Standard attention requires materializing the full $N \times N$ matrix. FlashAttention avoids this by processing K and V in tiles of size $B_c \times D$ and Q in tiles of size $B_r \times D$, maintaining per-row running statistics (m, ℓ) in memory:

- $m_i \in \mathbb{R}^{B_r}$: the running row-wise maximum of attention scores seen so far.
- $\ell_i \in \mathbb{R}^{B_r}$: the running row-wise sum of the unnormalized softmax exponentials.
- $O_i \in \mathbb{R}^{B_r \times D}$: the running output accumulator for those rows.

At each new tile, these are updated with a **rescaling** so that the final output equals the exact softmax without ever storing the full matrix.

The paper sets the on-chip SRAM budget to $M = 32,768$ floats (i.e. 32×1024). The formula from the paper is:

```

1 M  = 32 * 1024    # on-chip SRAM budget in number of floats
2 Bc = M // (4 * D)
3 Br = min(Bc, D)
4 Tc = (N + Bc - 1) // Bc    # number of K/V tiles
5 Tr = (N + Br - 1) // Br    # number of Q tiles

```

For $D = 64$: $B_c = 32768/256 = 128$ and $B_r = \min(128, 64) = 64$.

Your outer loop iterates over T_c tiles of K and V ; your inner loop iterates over T_r tiles of Q . For each (j, i) tile pair:

1. Load $K_j = K[jB_c : (j+1)B_c]$ and $V_j = V[jB_c : (j+1)B_c]$ (shape $B_c \times D$).
2. Load $Q_i = Q[iB_r : (i+1)B_r]$ (shape $B_r \times D$).
3. Compute tile scores: $S_{ij} = Q_i K_j^\top \cdot \text{scale}$ (shape $B_r \times B_c$).
4. Compute tile statistics: $\tilde{m}_{ij} = \max_{\text{row}}(S_{ij})$, $\tilde{P}_{ij} = \exp(S_{ij} - \tilde{m}_{ij})$, $\tilde{\ell}_{ij} = \sum_{\text{row}} \tilde{P}_{ij}$.
5. Update running statistics and output using the online softmax correction.

Part C: PyTorch Built-in FlashAttention

Use PyTorch's built-in fused attention kernel as a performance upper bound. No custom attention logic is required. Plot runtime vs. N alongside Parts A and B. Comment on: (1) at what N the built-in implementation begins to clearly outperform Part A, and (2) why it is so much faster than your Part B implementation.

Part D: FlashAttention Forward Pass in CUDA

Implement the FlashAttention forward pass as a custom CUDA kernel using shared memory tiling and on-register online softmax updates. The sequence dimension N is divided into fixed-size tiles:

$$\begin{aligned}
 B_r &= 32 \quad (\text{rows per } Q\text{-tile, controls the inner loop}), \\
 B_c &= 32 \quad (\text{rows per } K/V\text{-tile, controls the outer loop}).
 \end{aligned}$$

This gives

$$T_r = \left\lceil \frac{N}{B_r} \right\rceil \text{ row-tiles}, \quad T_c = \left\lceil \frac{N}{B_c} \right\rceil \text{ column-tiles}.$$

Concretely, for a single (b, h) slice:

- $Q_{b,h}$ is partitioned into T_r horizontal strips, each of shape $[B_r \times d]$. The i -th strip (0-indexed) covers rows $[i \cdot B_r, (i+1) \cdot B_r)$ of $Q_{b,h}$.
- $K_{b,h}$ and $V_{b,h}$ are each partitioned into T_c horizontal strips, each of shape $[B_c \times d]$. The j -th strip covers rows $[j \cdot B_c, (j+1) \cdot B_c)$.

At tile iteration (i, j) , the kernel computes the $[B_r \times B_c]$ block of attention logits between Q -tile i and K -tile j , and incrementally accumulates the corresponding $[B_r \times d]$ contribution into O -tile i . After all T_c outer iterations, O -tile i holds the exact same values as standard attention would produce for those B_r rows.

Grid and Block Decomposition: Who Does What

The kernel is launched with a 2-D grid of thread blocks:

```
dim3 grid(B, NH);    // total of B * NH thread blocks
dim3 block(Br);      // 32 threads per block
```

Each thread block is assigned one (**batch b , head h**) pair via

```
int b = blockIdx.x;   // batch index
int h = blockIdx.y;   // head index
```

That block is *solely* responsible for computing the entire output slice $O_{b,h} \in \mathbb{R}^{N \times d}$. No two blocks share any work or communicate; they each operate on a disjoint slice of Q , K , V , and O .

Threads within a block. Each block has $B_r = 32$ threads indexed by

```
int tx = threadIdx.x; // tx in {0, 1, ..., Br-1}
```

Within any given Q -tile i , **thread tx owns exactly one row**. This ownership is total—thread tx loads that row’s Q vector, computes all B_c dot products for that row, tracks that row’s running max and sum, and writes that row’s output back to global memory. No other thread ever reads or writes to those locations.

Thread tx also owns row tx of the current K/V -tile for the purposes of *loading*: thread tx is responsible for copying K -row $(j \cdot B_c + tx)$ and V -row $(j \cdot B_c + tx)$ into shared memory so that all 32 threads collectively fill the $[B_c \times d]$ tile in a single round of coalesced global loads.

Global memory base offsets. Because Q , K , and V are stored in row-major order as flat arrays of shape $[B \times N_H \times N \times d]$, the (b, h) slice begins at the flat offset:

```
int qkv_base = (b * NH + h) * N * d; // first float of Q_{b,h}
int lm_base  = (b * NH + h) * N;     // first float of m_{b,h} / l_{b,h}
```

Within that slice, the element at sequence position n and embedding index x is at `qkv_base + n*d + x`. Tile i of Q therefore starts at `qkv_base + (i*Br)*d`, and thread tx ’s row within that tile starts at `qkv_base + (i*Br + tx)*d`.

Shared Memory Layout

Declare a single `__shared__` array and manually partition it into four contiguous sub-arrays:

```
__shared__ float smem[];

float* Qi = smem;           // Q tile: Br x d floats
float* Kj = Qi + Br * d;    // K tile: Bc x d floats
float* Vj = Kj + Bc * d;    // V tile: Bc x d floats
float* S  = Vj + Bc * d;    // scores: Br x Bc floats
```

All four sub-arrays use **row-major** indexing, e.g. `Qi[tx*d + x]` is element (tx, x) of the Q tile. The total shared memory size passed at launch is:

```
size_t smem = (Br*d + 2*Bc*d + Br*Bc) * sizeof(float);
```

Outer Loop: Iterating Over K/V Tiles ($j = 0 \dots T_c - 1$)

The outer loop strides over the T_c column-tiles of K and V . **Every thread** in the block cooperates to load the current tile: thread $\mathbf{tx}$ loads row $\mathbf{tx}$ of the tile, i.e., global K/V row $j \cdot B_c + \mathbf{tx}$, into shared memory:

```
for (int j = 0; j < Tc; j++) {  
  
    for (int x = 0; x < d; x++) {  
        int kv_idx = qkv_base + (j * Bc + tx) * d + x;  
        Kj[tx * d + x] = K[kv_idx];    // thread tx writes its K row  
        Vj[tx * d + x] = V[kv_idx];    // thread tx writes its V row  
    }  
    __syncthreads();    // BARRIER 1: all Bc rows of K/V are in smem  
}
```

Inner Loop: Iterating Over Q Tiles ($i = 0 \dots T_r - 1$)

For each Q -tile i , thread $\mathbf{tx}$ independently owns and updates **one row** of the output. The five steps are:

Step 1 — Load the Q tile. Thread $\mathbf{tx}$ loads its own row from global memory into shared memory. The global sequence row is $i \cdot B_r + \mathbf{tx}$. No barrier is needed here because thread $\mathbf{tx}$ only ever reads from its own slot $Q_i[\mathbf{tx} \cdot d + *]$ in subsequent steps; no other thread accesses that slot.

Step 2 — Read running statistics. Thread $\mathbf{tx}$ reads the scalars m_{prev} and ℓ_{prev} from global buffers $\mathbf{m}[]$ and $\mathbf{l}[]$, initialized to $-\infty$ and 0, respectively, before the kernel launches.

Step 3 — Compute dot products and tile maximum. Thread $\mathbf{tx}$ computes B_c dot products: its Q row against each of the B_c K rows now resident in $\mathbf{Kj}$. All arithmetic is in registers; results are written into the $\mathbf{S}$ sub-array in row-major order. After this step, $\mathbf{S}[\mathbf{tx} \cdot B_c + 0 \dots B_c - 1]$ holds the scaled attention logits for thread $\mathbf{tx}$'s query row against all B_c key rows in the current tile and $\text{row_m} = \tilde{m}_j = \max_y S_{\mathbf{tx}, y}$.

Step 4 — Exponentiate and accumulate tile sum. Thread $\mathbf{tx}$ subtracts the tile maximum for numerical stability, exponentiates in-place, and accumulates the tile normalizer.

Step 5 — Online softmax update and output rescaling. This is the mathematical core of FlashAttention. Thread $\mathbf{tx}$ merges the tile statistics with the running statistics using the online-softmax recurrence, rescales the accumulated output, and adds the new $P \cdot V$ contribution; all in registers except for the final global writes. After all T_r inner iterations are complete, the second barrier fires before the next outer iteration.

Verification and Profiling

Verify your CUDA kernel against Parts A–C to within 10^{-3} maximum absolute difference. Profile your kernel using `ncu` or `nvprof` and report at least the achieved occupancy, shared memory usage per block, and global memory throughput. Plot runtime vs. N alongside the other implementations.

